



**Universidad del Sur**

## **DOCTORADO EN SISTEMAS COMPUTACIONALES**

### **ASIGNATURA:**

SEMINARIO DE DESARROLLO  
DE SISTEMAS DISTRIBUIDOS

### **ASESOR ACADÉMICO:**

DR. IVÁN ORTÍZ RAMIREZ

### **ACTIVIDAD:**

ACTIVIDAD 3. DESARROLLO DEL PROTOTIPO DEL  
PROYECTO INTEGRADOR

### **DOCTORANTES:**

PEDRO CRUZ RIVAS  
LUZ MARÍA MARQUEZ AGUNDEZ  
ILEANA BERENICE SANTIAGO GARCÍA  
JOSE EDUARDO CASTILLO MORALES

### **MODALIDAD:**

EN LINEA

22 de julio de 2026

|                                                                                                    |    |
|----------------------------------------------------------------------------------------------------|----|
| Organización del equipo .....                                                                      | 4  |
| • Nombre del proyecto .....                                                                        | 4  |
| • Objetivo general.....                                                                            | 4  |
| Revisión de la arquitectura .....                                                                  | 7  |
| • UML, Componentes, Interfaces, Tecnologías, Arquitectura distribuida, Flujo de comunicación ..... | 7  |
| Diagrama de casos de uso.....                                                                      | 7  |
| Diagrama de clases.....                                                                            | 7  |
| Diagrama de componentes .....                                                                      | 8  |
| Diagrama de despliegue.....                                                                        | 9  |
| Arquitectura C4 (Cono, Contenedores, Componentes).....                                             | 9  |
| Nivel 1 - Diagrama de Cono .....                                                                   | 9  |
| Nivel 2 - Diagrama de Contenedores .....                                                           | 10 |
| Nivel 3 — Diagrama de Componentes .....                                                            | 10 |
| • Justificación de cambios.....                                                                    | 10 |
| Configuración del entorno de desarrollo.....                                                       | 12 |
| Instalación y configuración de Visual Studio Code .....                                            | 12 |
| Instalación y configuración de Git.....                                                            | 12 |
| Configuración de la plataforma de alojamiento del repositorio (GitHub) .....                       | 13 |
| Instalación y configuración de Go .....                                                            | 14 |
| Instalación y configuración de Docker Engine.....                                                  | 14 |
| Verificación de Docker Compose .....                                                               | 15 |
| Creación de la estructura del proyecto .....                                                       | 15 |
| Creación de la estructura de carpetas.....                                                         | 15 |
| Creación del archivo Docker Compose .....                                                          | 16 |
| Configuración de PostgreSQL en Docker Compose .....                                                | 16 |
| Anexos (Git, script de BD, manual de instalación y manual de usuario) .....                        | 20 |
| Instalación y configuración de Visual Studio Code .....                                            | 24 |
| Instalación y configuración de Git.....                                                            | 26 |
| Configuración de la plataforma de alojamiento del repositorio (GitHub) .....                       | 29 |
| Instalación y configuración de OpenJDK 21 LTS .....                                                | 30 |
| Instalación y configuración de Apache Maven .....                                                  | 32 |
| Instalación y configuración de Go .....                                                            | 35 |

|                                                            |    |
|------------------------------------------------------------|----|
| Instalación y configuración de Docker Engine .....         | 37 |
| Verificación de Docker Compose .....                       | 39 |
| Creación de la estructura del proyecto .....               | 40 |
| Crear la estructura de carpetas .....                      | 40 |
| Creación del archivo Docker Compose .....                  | 41 |
| Configuración de PostgreSQL en Docker Compose .....        | 43 |
| Configuración de ZooKeeper en Docker Compose .....         | 46 |
| Configuración de Apache Kafka en Docker Compose .....      | 48 |
| Configuración de Kong Gateway en Docker Compose .....      | 49 |
| Despliegue de la infraestructura con Docker Compose .....  | 51 |
| Integración de los microservicios .....                    | 53 |
| Verificación de la comunicación entre microservicios ..... | 55 |
| Conclusiones.....                                          | 57 |

## Organización del equipo

- **Nombre del proyecto**  
**Sistema de gestión hospitalaria**
- **Objetivo general**

El objetivo principal del Sistema de Gestión Hospitalaria es optimizar la administración de los servicios de salud en hospitales públicos mediante la automatización e integración de los procesos clínicos y administrativos. Con este sistema se busca reducir los tiempos de espera de los pacientes, mejorar el acceso a la información médica, facilitar la coordinación entre las diferentes áreas hospitalarias, garantizar la disponibilidad de los expedientes clínicos electrónicos y fortalecer la toma de decisiones mediante información confiable y actualizada.

- **Roles y responsabilidades**

| <b>Integrante</b>                      | <b>Rol</b>                                                  | <b>Responsabilidades</b>                                                                                                                                                                                                                                    |
|----------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pedro Cruz Rivas</b>                | <b>Líder Técnico</b>                                        | Coordinar las actividades del equipo, supervisar el cumplimiento del cronograma, validar la arquitectura del sistema, revisar la integración de los microservicios y asegurar que todos los módulos funcionen correctamente.                                |
| <b>Luz María Márquez Agundez</b>       | <b>Desarrolladora Backend</b>                               | Desarrollar la lógica de negocio de los microservicios, implementar la comunicación con las bases de datos, crear las API REST y colaborar en la integración entre servicios.                                                                               |
| <b>Ileana Berenice Santiago García</b> | <b>Desarrolladora Frontend e Integración</b>                | Diseñar e implementar la interfaz de usuario, integrar el frontend con los microservicios mediante las API, realizar pruebas de integración y verificar la correcta comunicación entre los componentes del sistema.                                         |
| <b>José Eduardo Castillo Morales</b>   | <b>Administrador del Repositorio y Control de Versiones</b> | Configurar el repositorio en GitHub, administrar las ramas del proyecto (main, develop y ramas por funcionalidad), apoyar en la integración del código, documentar los cambios y mantener actualizada la documentación técnica y los manuales del proyecto. |

- Cronograma

| Fase          | Actividad                                       | Responsable           | Duración | Inicio     | Fin        |
|---------------|-------------------------------------------------|-----------------------|----------|------------|------------|
| Planeación    | Definir nombre del proyecto, objetivo y alcance | Todo el equipo        | 5 días   | 20/07/2026 | 24/07/2026 |
|               | Asignación de roles y responsabilidades         | Pedro (Líder Técnico) | 2 días   | 27/07/2026 | 28/07/2026 |
|               | Elaboración del cronograma                      | Pedro                 | 3 días   | 29/07/2026 | 31/07/2026 |
| Análisis      | Revisión de diagramas UML                       | Todo el equipo        | 4 días   | 03/08/2026 | 06/08/2026 |
|               | Revisión de componentes e interfaces            | Backend y Frontend    | 3 días   | 07/08/2026 | 11/08/2026 |
|               | Validación de la arquitectura distribuida       | Todo el equipo        | 4 días   | 12/08/2026 | 17/08/2026 |
|               | Ajustes y justificación de cambios              | Pedro                 | 3 días   | 18/08/2026 | 20/08/2026 |
| Configuración | Instalación del entorno de desarrollo           | Todo el equipo        | 5 días   | 21/08/2026 | 27/08/2026 |
|               | Configuración de Git y GitHub                   | José Eduardo          | 3 días   | 28/08/2026 | 01/09/2026 |
|               | Creación del repositorio                        | José Eduardo          | 2 días   | 02/09/2026 | 03/09/2026 |

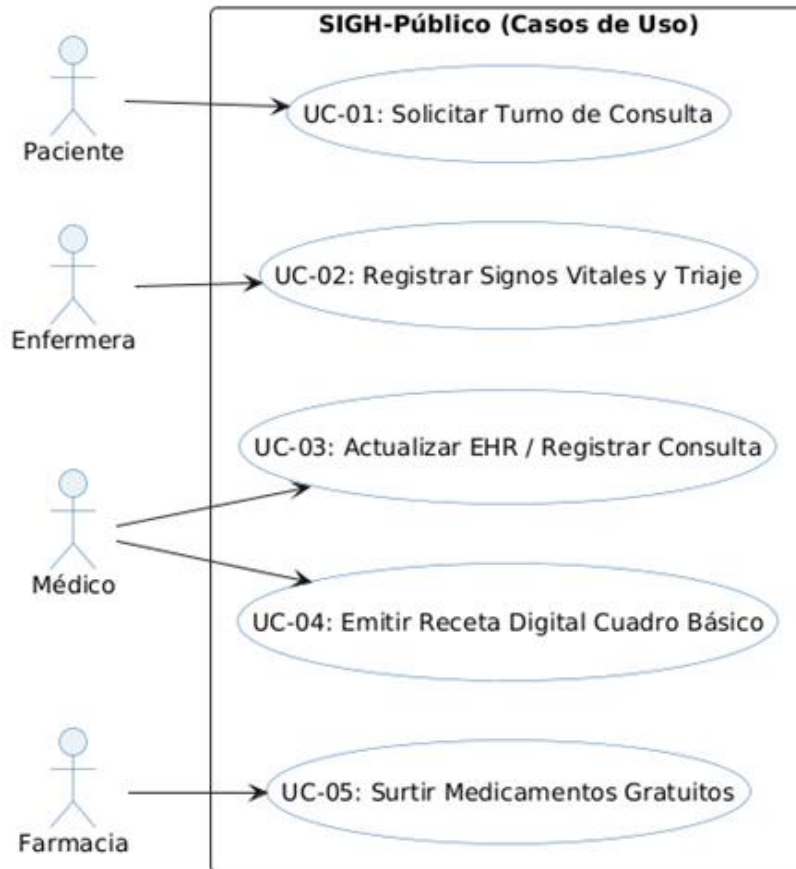
|                            |                                              |                    |         |            |            |
|----------------------------|----------------------------------------------|--------------------|---------|------------|------------|
|                            | Definición de ramas y estructura de carpetas | José Eduardo       | 3 días  | 04/09/2026 | 08/09/2026 |
| <b>Desarrollo Backend</b>  | Desarrollo del MS Turnos                     | Luz María          | 10 días | 09/09/2026 | 22/09/2026 |
|                            | Desarrollo del MS Consulta                   | Luz María          | 10 días | 23/09/2026 | 06/10/2026 |
|                            | Desarrollo del MS EHR                        | Pedro              | 8 días  | 07/10/2026 | 16/10/2026 |
|                            | Desarrollo del MS Farmacia                   | Pedro              | 8 días  | 19/10/2026 | 28/10/2026 |
| <b>Desarrollo Frontend</b> | Desarrollo de interfaces                     | Ileana             | 15 días | 29/10/2026 | 18/11/2026 |
|                            | Integración Frontend-Backend                 | Ileana y Luz María | 10 días | 19/11/2026 | 02/12/2026 |
| <b>Pruebas</b>             | Pruebas funcionales                          | Todo el equipo     | 7 días  | 03/12/2026 | 11/12/2026 |
|                            | Corrección de errores                        | Todo el equipo     | 5 días  | 14/12/2026 | 18/12/2026 |
| <b>Documentación</b>       | Manual de instalación                        | José Eduardo       | 3 días  | 21/12/2026 | 23/12/2026 |
|                            | Manual de usuario                            | Ileana             | 3 días  | 21/12/2026 | 23/12/2026 |
|                            | Script de base de datos                      | Luz María          | 2 días  | 24/12/2026 | 25/12/2026 |
|                            | Conclusiones y entrega final                 | Todo el equipo     | 4 días  | 28/12/2026 | 31/12/2026 |

- Herramientas de comunicación

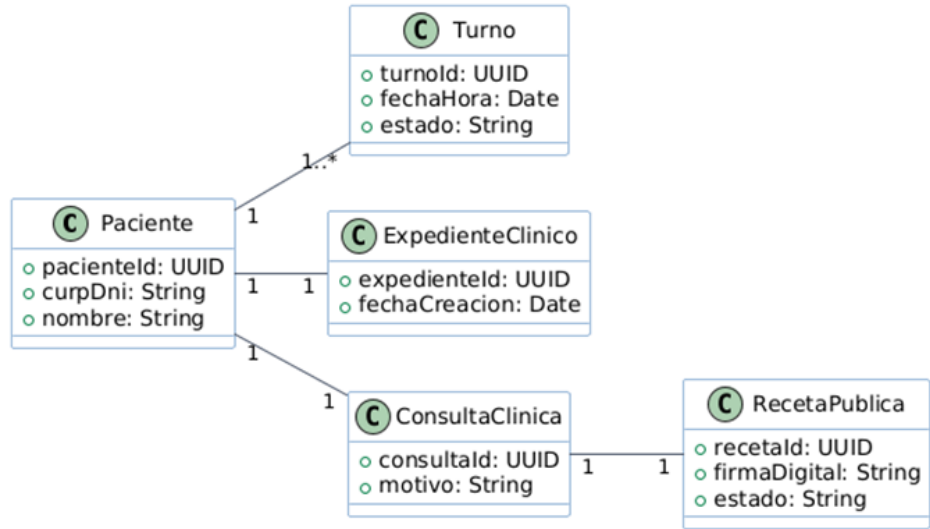
## Revisión de la arquitectura

- UML, Componentes, Interfaces, Tecnologías, Arquitectura distribuida, Flujo de comunicación

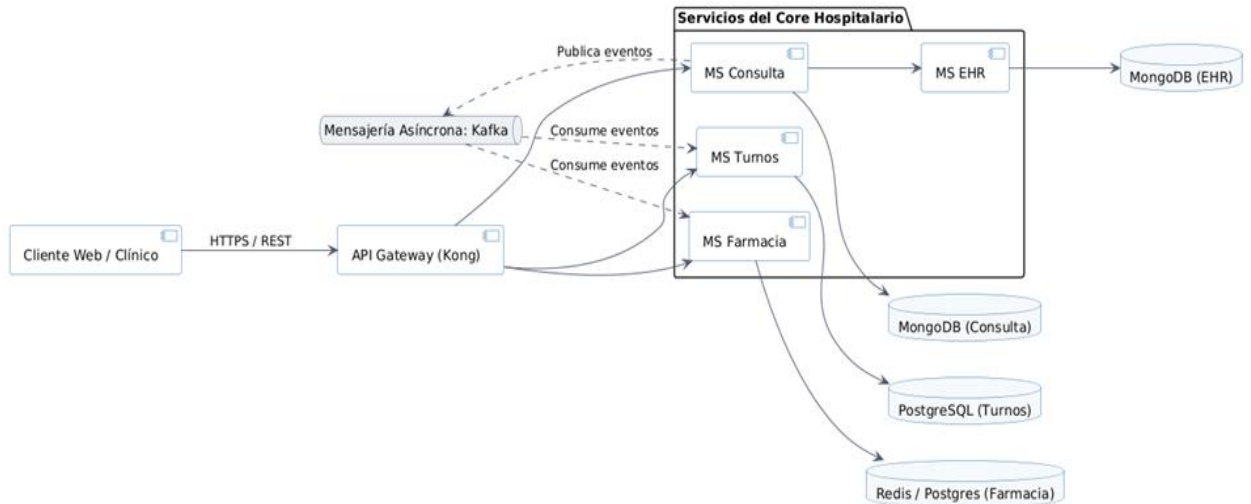
### Diagrama de casos de uso



### Diagrama de clases

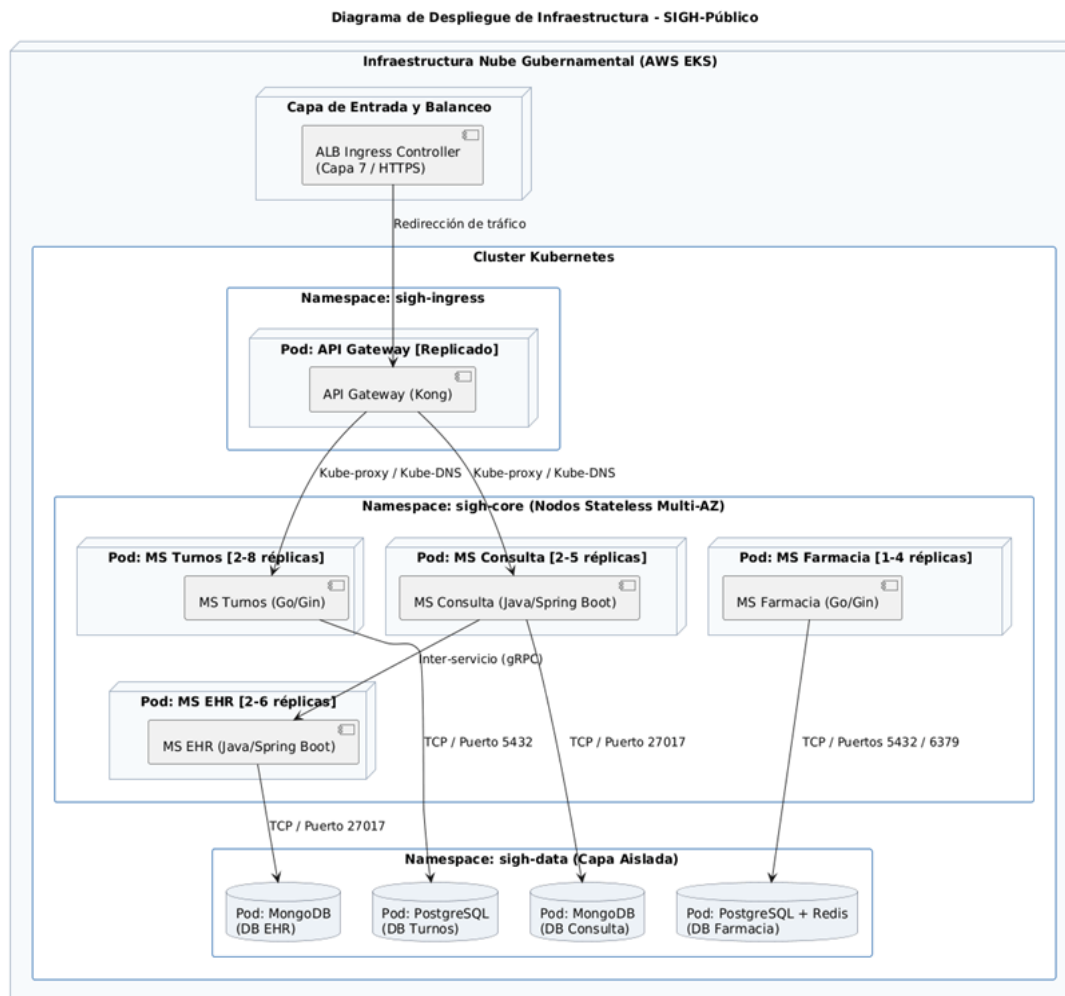


## Diagrama de componentes





## Diagrama de despliegue



## Arquitectura C4 (Cono, Contenedores, Componentes)

### Nivel 1 - Diagrama de Cono

El SIGH-Público interactúa en el centro del ecosistema asistencial. El SIGH-Público interactúa en el centro del ecosistema asistencial. Los médicos, enfermeras y administrativos del hospital público utilizan terminales web conectadas a la red local segura. El sistema se integra externamente con el Registro Nacional de Identidad para validar CURP y con el Sistema Integrado de Salud del Estado para reportar indicadores epidemiológicos.

## Nivel 2 - Diagrama de Contenedores

- Desarrollo de una Aplicación Médica de Escritorio/Web: Interfaz unificada utilizada en consultorios y en las estaciones de enfermería.
- API Gateway (Kong): Gestiona las peticiones de red internas, control de flujo e inyección del correlation-ID de seguimiento de pacientes.
- Microservicios Desacoplados: Turnos, Consulta, Farmacia y EHR, corriendo en contenedores Docker autónomos.
- Bases de datos políglotas: Motores de persistencia independientes por microservicio, evitando acoplamientos lógicos.

## Nivel 3 — Diagrama de Componentes

El core microservicio de Consulta está organizado bajo la Arquitectura Hexagonal:

- Adaptadores primarios (Adaptadores de entrada): Controladores REST para peticiones HTTP/JSON y gRPC de triaje.
- Core de Aplicación: Servicios que implementan las invariantes clínicas del flujo de atención.
- Adaptadores de Salida: Repositorio de datos de MongoDB, adaptadores para publicar eventos en Kafka (por ejemplo, RecetaEmitidaEvent) y cliente gRPC para hablar con el expediente.

- **Justificación de cambios**

| Criterio      | Justificación                                                                                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Escalabilidad | Permite el escalado independiente de los servicios con alta demanda transaccional (ej. Triage o Turnos) sin desperdiciar recursos de infraestructura pública en servidores. |

|                            |                                                                                                                                                                                                |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rendimiento                | Uso de tecnologías y bases de datos óptimas para cada tarea (ej. gRPC de ultra baja latencia para comunicación interna y MongoDB para formato clínico flexible).                               |
| Facilidad de mantenimiento | Equipos de TI pequeños del sector público pueden mantener y desplegar componentes individuales sin necesidad de comprender o compilar todo el sistema hospitalario.                            |
| Disponibilidad             | Si el servicio de farmacia o el de notificaciones se degrada, los servicios críticos de triaje e historial clínico (EHR) continúan operando de manera ininterrumpida.                          |
| Costos de implementación   | Aunque la arquitectura distribuida tiene costos iniciales de configuración en red, a largo plazo optimiza el consumo de servidores de cómputo públicos al apagar o reducir réplicas inactivas. |

# Configuración del entorno de desarrollo

La configuración del entorno de desarrollo corresponde al conjunto de herramientas y tecnologías necesarias para implementar el Sistema de Gestión Hospitalaria. Debido a que el proyecto está basado en una arquitectura de microservicios, fue necesario seleccionar herramientas que permitieran desarrollar, ejecutar, integrar y administrar cada uno de los servicios de forma independiente, manteniendo una infraestructura organizada y fácil de reproducir.

El entorno de desarrollo está conformado por Visual Studio Code como entorno de desarrollo integrado, Git como sistema de control de versiones, GitHub como plataforma de alojamiento del repositorio, OpenJDK 21 LTS y Apache Maven 3.9.x para el desarrollo de los microservicios implementados con Spring Boot, Go 1.24.x para los servicios desarrollados con el framework Gin, Docker Engine 28.x y Docker Compose v2 para la contenedorización de la aplicación. Asimismo, la infraestructura incorpora PostgreSQL 17, Apache ZooKeeper 3.9, Apache Kafka 3.9 y Kong Gateway 3.8, los cuales permiten implementar la arquitectura definida para el proyecto.

La combinación de estas tecnologías proporciona un entorno estable y compatible para el desarrollo, integración y ejecución del Sistema de Gestión Hospitalaria. El procedimiento detallado de instalación y configuración de cada una de ellas se presenta en el Anexo A. Manual de instalación.

## Instalación y configuración de Visual Studio Code

Para el desarrollo del Sistema de Gestión Hospitalaria se utilizará Visual Studio Code, en su versión estable, como entorno de desarrollo integrado (IDE). Esta herramienta fue seleccionada por su compatibilidad con Spring Boot, Go, Docker y Git, además de ofrecer un entorno ligero y extensible mediante complementos oficiales.

Visual Studio Code permitirá desarrollar y administrar el código fuente de los microservicios, editar archivos de configuración como `docker-compose.yml` y facilitar tareas de depuración y control de versiones desde una única aplicación. El procedimiento de instalación y configuración se presenta en el Anexo A. Manual de instalación.

## Instalación y configuración de Git

Para administrar el código fuente del proyecto se utilizará Git, en su versión 2.49.x, como sistema de control de versiones. Esta herramienta permitirá registrar el historial de cambios realizados durante el desarrollo, administrar diferentes versiones del software y facilitar el trabajo organizado sobre cada uno de los microservicios.

La integración de Git con Visual Studio Code y GitHub permitirá mantener un control adecuado de la evolución del Sistema de Gestión Hospitalaria durante todas las etapas del desarrollo. El procedimiento de instalación y configuración se encuentra documentado en el Anexo A. Manual de instalación.

### **Configuración de la plataforma de alojamiento del repositorio (GitHub)**

Como complemento al sistema de control de versiones se utilizará GitHub, plataforma de alojamiento remoto basada en Git. Su función principal será almacenar el código fuente del Sistema de Gestión Hospitalaria, mantener un respaldo permanente del proyecto y facilitar la sincronización de los cambios realizados por medio de Git.

La utilización de GitHub permitirá centralizar el desarrollo de los microservicios, mantener un historial de versiones y facilitar la colaboración durante el desarrollo del sistema. La configuración del repositorio se describe en el Anexo A. Manual de instalación.

### **Instalación y configuración de OpenJDK 21 LTS**

Para el desarrollo de los microservicios implementados con Spring Boot se utilizará OpenJDK 21 LTS (Long Term Support), una distribución de código abierto del Kit de Desarrollo de Java (JDK). Esta herramienta proporciona el compilador, la Máquina Virtual de Java (JVM) y las bibliotecas necesarias para desarrollar, ejecutar y mantener aplicaciones empresariales basadas en este lenguaje.

La elección de la versión 21 LTS se debe a que ofrece soporte extendido, mayor estabilidad y mejoras en el rendimiento respecto a versiones anteriores, siendo compatible con las versiones recientes de Spring Boot utilizadas en el proyecto. Gracias a ello, los microservicios MS Consulta y MS EHR podrán ejecutarse de forma segura y eficiente dentro de la arquitectura del Sistema de Gestión Hospitalaria.

El procedimiento de instalación y configuración de OpenJDK 21 LTS se presenta en el Anexo A. Manual de instalación.

### **Instalación y configuración de Apache Maven**

Para la administración de los proyectos desarrollados con Spring Boot se empleará Apache Maven 3.9.x, una herramienta orientada a la gestión de dependencias y automatización de la construcción de aplicaciones Java. Maven permite organizar la estructura del proyecto, descargar automáticamente las bibliotecas requeridas y simplificar procesos como la compilación, ejecución de pruebas y generación de archivos ejecutables.

Su utilización facilita el mantenimiento del código fuente y garantiza que todos los microservicios desarrollados en Java utilicen las mismas versiones de las dependencias, reduciendo problemas de compatibilidad durante el desarrollo y despliegue del sistema.

El procedimiento de instalación y configuración de Apache Maven se encuentra documentado en el Anexo A. Manual de instalación.

## **Instalación y configuración de Go**

Los microservicios MS Turnos y MS Farmacia serán desarrollados utilizando Go 1.24.x, un lenguaje de programación diseñado para crear aplicaciones de alto rendimiento y baja complejidad. Go se caracteriza por su velocidad de compilación, manejo eficiente de la concurrencia y reducido consumo de recursos, características que lo convierten en una opción adecuada para arquitecturas basadas en microservicios.

La incorporación de Go dentro del Sistema de Gestión Hospitalaria permite diversificar las tecnologías utilizadas según las necesidades de cada servicio, aprovechando su capacidad para atender múltiples solicitudes simultáneamente y mantener tiempos de respuesta bajos. Además, su integración con el framework Gin facilita la construcción de servicios REST de forma rápida y organizada.

El procedimiento de instalación y configuración de Go se presenta en el Anexo A. Manual de instalación.

## **Instalación y configuración de Docker Engine**

Para la ejecución de los microservicios y los servicios de infraestructura se utilizará Docker Engine 28.x, una plataforma de contenedorización que permite empaquetar aplicaciones junto con todas sus dependencias dentro de contenedores independientes. Esta tecnología elimina las diferencias entre los entornos de desarrollo y ejecución, asegurando que el sistema funcione de la misma manera en cualquier equipo.

En el Sistema de Gestión Hospitalaria, Docker permitirá ejecutar de forma aislada los microservicios desarrollados con Spring Boot y Go, así como los componentes de infraestructura como PostgreSQL, Apache Kafka, ZooKeeper y Kong Gateway. Esta estrategia facilita el despliegue del sistema, simplifica su mantenimiento y mejora la escalabilidad de la arquitectura.

El procedimiento de instalación y configuración de Docker Engine se presenta en el Anexo A. Manual de instalación.

## **Verificación de Docker Compose**

Para la administración conjunta de todos los contenedores se utilizará Docker Compose v2, herramienta que permite definir y ejecutar múltiples servicios mediante un único archivo de configuración denominado `docker-compose.yml`. A través de este archivo se especifican las imágenes, redes, volúmenes y configuraciones necesarias para cada componente del sistema.

La utilización de Docker Compose simplifica el despliegue de la infraestructura del Sistema de Gestión Hospitalaria, ya que permite iniciar todos los microservicios y servicios de apoyo mediante un solo comando, garantizando que cada componente se ejecute con la configuración previamente establecida.

El procedimiento de instalación y verificación de Docker Compose se presenta en el Anexo A. Manual de instalación.

## **Creación de la estructura del proyecto**

Una vez configurado el entorno de desarrollo, fue necesario definir una estructura organizada para el Sistema de Gestión Hospitalaria. Debido a que el proyecto se implementa mediante una arquitectura de microservicios, cada componente debe mantenerse independiente para facilitar su desarrollo, mantenimiento y escalabilidad.

La estructura del proyecto se organizó en directorios específicos para cada microservicio, archivos de configuración, infraestructura y recursos compartidos. Esta distribución permite separar las responsabilidades de cada componente, mantener un código más ordenado y facilitar la incorporación de nuevas funcionalidades sin afectar el funcionamiento del resto del sistema.

El procedimiento para la creación de la estructura del proyecto se presenta en el Anexo A. Manual de instalación.

## **Creación de la estructura de carpetas**

La estructura de carpetas fue diseñada para organizar de manera lógica todos los elementos que conforman el Sistema de Gestión Hospitalaria. Cada microservicio dispone de su propio directorio, donde se almacenan el código fuente, los archivos de configuración, las dependencias y los recursos necesarios para su funcionamiento.

Además de los directorios destinados a los microservicios, se incluyen carpetas para la infraestructura, la documentación y otros recursos compartidos, permitiendo mantener una separación clara entre los distintos componentes del proyecto. Esta

organización facilita la administración del código, mejora la comprensión de la arquitectura y simplifica las tareas de mantenimiento durante el desarrollo.

La creación detallada de la estructura de carpetas se describe en el Anexo A. Manual de instalación.

### **Creación del archivo Docker Compose**

Para administrar la infraestructura del Sistema de Gestión Hospitalaria se utilizará el archivo `docker-compose.yml`, el cual constituye el punto central de configuración de todos los contenedores que forman parte del proyecto. En este archivo se definen los servicios que serán ejecutados, las redes de comunicación, los volúmenes de almacenamiento y las variables de entorno necesarias para cada componente.

La utilización de Docker Compose permite gestionar toda la infraestructura mediante una única configuración, simplificando el despliegue de los microservicios y de los servicios de apoyo. Esto facilita la reproducción del entorno de desarrollo en diferentes equipos y garantiza que todos los componentes utilicen la misma configuración durante las etapas de desarrollo y pruebas.

El procedimiento para la creación y configuración del archivo `docker-compose.yml` se presenta en el Anexo A. Manual de instalación.

### **Configuración de PostgreSQL en Docker Compose**

Como parte de la infraestructura del Sistema de Gestión Hospitalaria se incorporará PostgreSQL 17, un sistema de gestión de bases de datos relacional reconocido por su estabilidad, seguridad y cumplimiento de las propiedades ACID. Su función será almacenar la información estructurada del sistema, como los datos de pacientes, médicos, consultas, expedientes clínicos y demás registros que requieren integridad y consistencia.

La configuración de PostgreSQL dentro de Docker Compose permitirá ejecutar la base de datos como un contenedor independiente, facilitando su administración y garantizando que todos los microservicios autorizados puedan acceder a la información mediante una configuración centralizada. Esta estrategia mejora la portabilidad del proyecto y simplifica el despliegue del entorno de desarrollo.

El procedimiento de configuración de PostgreSQL en Docker Compose se encuentra documentado en el Anexo A. Manual de instalación.



## **Configuración de ZooKeeper en Docker Compose**

Para garantizar el correcto funcionamiento de la plataforma de mensajería utilizada en el Sistema de Gestión Hospitalaria se incorporará Apache ZooKeeper 3.9 como servicio de coordinación. ZooKeeper es una herramienta diseñada para administrar la configuración, sincronización y disponibilidad de sistemas distribuidos, permitiendo que los diferentes componentes trabajen de forma organizada y consistente.

En este proyecto, ZooKeeper será utilizado para coordinar el funcionamiento de Apache Kafka, gestionando la información necesaria para la comunicación entre los brokers y asegurando la estabilidad del servicio de mensajería. Su implementación mediante Docker Compose permite integrarlo fácilmente con el resto de la infraestructura, simplificando su administración y despliegue.

El procedimiento para la configuración de ZooKeeper en Docker Compose se presenta en el Anexo A. Manual de instalación.

## **Configuración de Apache Kafka en Docker Compose**

La arquitectura del Sistema de Gestión Hospitalaria requiere que algunos microservicios intercambien información sin depender de una comunicación directa entre ellos. Para ello se utilizará Apache Kafka 3.9, una plataforma de mensajería distribuida basada en eventos que permite la comunicación asíncrona entre aplicaciones.

Kafka facilitará el intercambio de información entre los microservicios cuando se generen eventos importantes, como el registro de una consulta médica, la actualización de un expediente clínico o la disponibilidad de medicamentos. Este mecanismo reduce el acoplamiento entre los servicios, mejora la escalabilidad de la aplicación y permite que cada microservicio procese la información de manera independiente.

La configuración de Kafka mediante Docker Compose permite integrarlo con el resto de la infraestructura del proyecto, asegurando una comunicación eficiente entre los componentes del sistema. El procedimiento detallado se encuentra documentado en el Anexo A. Manual de instalación.

## **Configuración de Kong Gateway en Docker Compose**

Para administrar el acceso a los microservicios se implementará Kong Gateway 3.8, una plataforma que actuará como punto de entrada único para las solicitudes realizadas por los clientes del sistema. Su principal función consiste en recibir las peticiones, dirigirlas al microservicio correspondiente y centralizar la administración del tráfico de la aplicación.

La incorporación de Kong Gateway mejora la organización de la arquitectura, ya que evita que los clientes se comuniquen directamente con cada microservicio. Además, proporciona una base para incorporar funcionalidades como autenticación, autorización, limitación de solicitudes y monitoreo del tráfico, fortaleciendo la seguridad y el rendimiento del Sistema de Gestión Hospitalaria.

El procedimiento para la configuración de Kong Gateway en Docker Compose se presenta en el Anexo A. Manual de instalación.

## **Despliegue de la infraestructura con Docker Compose**

Una vez definidos todos los servicios que conforman la infraestructura, se realizará su despliegue mediante Docker Compose v2. Esta herramienta permite iniciar de forma coordinada los contenedores correspondientes a PostgreSQL, ZooKeeper, Apache Kafka, Kong Gateway y los diferentes microservicios que integran el Sistema de Gestión Hospitalaria.

La utilización de Docker Compose simplifica la administración del entorno de desarrollo, ya que todos los servicios pueden iniciarse, detenerse o actualizarse mediante una única configuración. Esto garantiza que la infraestructura se mantenga consistente en cada ejecución y facilita la reproducción del entorno en distintos equipos de trabajo.

El procedimiento de despliegue de la infraestructura se describe en el Anexo A. Manual de instalación.

## **Integración de los microservicios**

Después del despliegue de la infraestructura, se llevará a cabo la integración de los microservicios que conforman el Sistema de Gestión Hospitalaria. Esta etapa tiene como objetivo verificar que cada servicio pueda comunicarse correctamente con las bases de datos, los servicios de mensajería y el API Gateway, conforme a la arquitectura definida para el proyecto.

La integración contempla la comunicación mediante servicios REST, el intercambio de información utilizando gRPC cuando sea necesario y la comunicación asíncrona mediante Apache Kafka, permitiendo que los microservicios operen de manera coordinada y mantengan un bajo nivel de dependencia entre sí.

El procedimiento detallado para la integración de los microservicios se presenta en el Anexo A. Manual de instalación.

## Verificación de la comunicación entre microservicios

Como etapa final de la configuración del entorno de desarrollo, se realizará la verificación de la comunicación entre los diferentes microservicios y los servicios de infraestructura. Esta actividad tiene como propósito comprobar que las solicitudes, respuestas y eventos sean procesados correctamente, asegurando que cada componente interactúe conforme a la arquitectura diseñada.

La verificación permitirá confirmar el correcto funcionamiento de las conexiones con PostgreSQL, la comunicación mediante REST y gRPC, el intercambio de eventos a través de Apache Kafka y el enrutamiento de solicitudes realizado por Kong Gateway. Con ello se garantiza que el entorno de desarrollo se encuentre preparado para iniciar las pruebas funcionales y continuar con las siguientes fases del proyecto.

El procedimiento de verificación se encuentra documentado en el Anexo A. Manual de instalación.

| Herramienta        | Versión        | Propósito                                |
|--------------------|----------------|------------------------------------------|
| Ubuntu Server      | 24.04.2 LTS    | Sistema operativo                        |
| Visual Studio Code | Stable         | Entorno de desarrollo integrado          |
| Git                | 2.49.x         | Control de versiones                     |
| GitHub             | Plataforma web | Repositorio remoto                       |
| OpenJDK            | 21 LTS         | Desarrollo de microservicios Spring Boot |
| Apache Maven       | 3.9.x          | Gestión de dependencias y compilación    |
| Go                 | 1.24.x         | Desarrollo de microservicios con Gin     |
| Docker Engine      | 28.x           | Contenedorización                        |
| Docker Compose     | v2             | Orquestación de contenedores             |
| PostgreSQL         | 17             | Base de datos relacional                 |
| Apache ZooKeeper   | 3.9            | Coordinación de Kafka                    |
| Apache Kafka       | 3.9            | Mensajería entre microservicios          |
| Kong Gateway       | 3.8            | API Gateway                              |

## Anexos (Git, script de BD, manual de instalación y manual de usuario)

Con base en la arquitectura definida en la Actividad 2, el Sistema de Gestión Hospitalaria será implementado utilizando un conjunto de tecnologías que permiten desarrollar una solución distribuida basada en microservicios. Estas herramientas fueron seleccionadas para satisfacer los requerimientos de escalabilidad, disponibilidad, mantenibilidad y comunicación entre servicios establecidos durante el análisis y diseño del sistema. Asimismo, proporcionan el entorno necesario para desarrollar, ejecutar, contenerizar y administrar cada uno de los componentes que integran la plataforma hospitalaria.

La siguiente tabla presenta las tecnologías que conforman el entorno de desarrollo y que serán instaladas y configuradas durante esta actividad.

| <b>Categoría</b>         | <b>Tecnología</b>         | <b>Función dentro del proyecto</b>                                                      |
|--------------------------|---------------------------|-----------------------------------------------------------------------------------------|
| Sistema Operativo        | Ubuntu Server 24.04.2 LTS | Plataforma base donde se instalarán y ejecutarán todos los componentes del sistema.     |
| Control de versiones     | Git                       | Administración del código fuente y control de cambios del proyecto.                     |
| Repositorio remoto       | GitHub                    | Almacenamiento centralizado del código y colaboración entre los integrantes del equipo. |
| Lenguaje de programación | OpenJDK 21 LTS            | Entorno de ejecución para los microservicios desarrollados con Java.                    |
| Gestor de dependencias   | Apache Maven 3.9.11       | Compilación y administración de dependencias de los proyectos Spring Boot.              |
| Framework                | Spring Boot 3.5.4         | Desarrollo de los microservicios MS Consulta y MS EHR.                                  |
| Lenguaje de programación | Go 1.24.5                 | Desarrollo de los microservicios MS Turnos y MS Farmacia.                               |
| Framework                | Gin 1.10.1                | Framework HTTP utilizado por los microservicios desarrollados en Go.                    |
| Editor de código         | Visual Studio Code 1.102  | Desarrollo y mantenimiento del código fuente del proyecto.                              |
| Contenedores             | Docker Engine 28.3.3      | Ejecución aislada de cada microservicio y de la infraestructura del sistema.            |
| Orquestación local       | Docker Compose 2.38.2     | Levantamiento del entorno completo de desarrollo mediante contenedores.                 |

|                                  |                          |                                                                                                       |
|----------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------|
| Base de datos relacional         | PostgreSQL 17.5          | Almacenamiento de la información transaccional de los microservicios que requieren consistencia ACID. |
| Base de datos NoSQL              | MongoDB Community 8.0.12 | Almacenamiento de expedientes clínicos y documentos con estructura flexible.                          |
| Caché distribuida                | Redis 8.0.3              | Almacenamiento temporal de información de acceso frecuente para mejorar el rendimiento.               |
| Mensajería                       | Apache Kafka 4.0.0       | Comunicación asíncrona basada en eventos entre los microservicios.                                    |
| Coordinación de Kafka            | Apache ZooKeeper 3.9.3   | Administración y coordinación del clúster de Kafka de acuerdo con la arquitectura propuesta.          |
| API Gateway                      | Kong Gateway OSS 3.10.0  | Punto único de acceso para las solicitudes dirigidas a los microservicios.                            |
| Comunicación síncrona            | REST                     | Exposición de servicios hacia clientes externos y comunicación entre componentes.                     |
| Comunicación de alto rendimiento | gRPC                     | Comunicación interna de baja latencia entre microservicios.                                           |
| Seguridad                        | JWT                      | Autenticación y validación de identidad de los usuarios.                                              |
| Autorización                     | OAuth 2.0                | Gestión segura del acceso a los servicios expuestos mediante el API Gateway.                          |

Este conjunto de tecnologías corresponde al entorno sobre el cual se desarrollará e implementará el Sistema de Gestión Hospitalaria, manteniendo coherencia con la arquitectura distribuida basada en microservicios definida en la Actividad 2, donde se establecen los lenguajes, frameworks, mecanismos de comunicación, bases de datos, API Gateway, mensajería y estrategia de contenerización que conforman la solución

El primer paso para implementar el prototipo del Sistema de Gestión Hospitalaria consiste en preparar el entorno sobre el cual se ejecutarán todas las herramientas de desarrollo, los microservicios y los servicios de infraestructura definidos en la arquitectura propuesta durante la Actividad 2.

Para este proyecto se utilizará **Ubuntu Server 24.04.2 LTS (Long Term Support)** como sistema operativo principal. Esta distribución Linux fue seleccionada debido a su estabilidad, amplio soporte por parte de la comunidad y compatibilidad con tecnologías como Docker, Java, Go, PostgreSQL, MongoDB, Redis, Apache Kafka y Kong Gateway. Asimismo, al tratarse de una versión LTS, proporciona actualizaciones de seguridad y mantenimiento durante un periodo prolongado, lo que la convierte en una alternativa adecuada para ambientes de desarrollo y pruebas.

La instalación del sistema operativo constituye la base del entorno de desarrollo, ya que sobre él se instalarán posteriormente todas las herramientas necesarias para construir y ejecutar el prototipo basado en microservicios.

### ¿Qué se instalará en esta sección?

En esta primera etapa únicamente se instalará el sistema operativo.

| Componente                | Descripción                                                                    |
|---------------------------|--------------------------------------------------------------------------------|
| Ubuntu Server 24.04.2 LTS | Sistema operativo que alojará todas las herramientas y servicios del proyecto. |

### ¿Por qué se eligió Ubuntu Server?

Ubuntu Server ofrece múltiples ventajas para este proyecto:

- Es un sistema operativo de código abierto ampliamente utilizado en entornos empresariales.
- Posee compatibilidad con todas las tecnologías utilizadas en la arquitectura del proyecto.
- Cuenta con una gran cantidad de documentación oficial.
- Facilita la administración desde consola.
- Tiene soporte para Docker y Kubernetes.
- Presenta un excelente rendimiento para arquitecturas de microservicios.
- Es estable y seguro gracias a su versión LTS (Long Term Support).

## Requisitos mínimos

Antes de comenzar la instalación es recomendable verificar que el equipo cumpla con las siguientes características.

| Recurso             | Recomendado              |
|---------------------|--------------------------|
| Procesador          | 4 núcleos                |
| Memoria RAM         | 8 GB (16 GB recomendado) |
| Almacenamiento      | 100 GB SSD               |
| Arquitectura        | x64                      |
| Conexión a Internet | Sí                       |

## Actualizar el índice de paquetes

Antes de instalar cualquier software es recomendable actualizar la información de los repositorios configurados en Ubuntu.

Ejecutar el siguiente comando:

```
sudo apt update
```

### ¿Qué hace este comando?

- **sudo**: ejecuta el comando con privilegios de administrador.
- **apt**: gestor de paquetes de Ubuntu.
- **update**: descarga la lista más reciente de paquetes disponibles desde los repositorios configurados.

### ¿Por qué se ejecuta?

Este comando **no instala software**, únicamente sincroniza la información de los repositorios para asegurar que las instalaciones posteriores utilicen las versiones más recientes disponibles.

## Paso 2. Actualizar el sistema operativo

Una vez actualizada la lista de paquetes, se procede a instalar las actualizaciones disponibles.

Ejecutar:

```
sudo apt upgrade -y
```

### ¿Qué hace este comando?

Instala todas las actualizaciones disponibles para los paquetes ya instalados en el sistema operativo.

### Explicación del comando

- **upgrade**: actualiza todos los paquetes instalados.
- **-y**: responde automáticamente "Sí" a las preguntas del instalador.

### ¿Por qué se ejecuta?

Permite trabajar sobre un sistema operativo actualizado, reduciendo problemas de compatibilidad y mejorando la estabilidad del entorno de desarrollo.

## Paso 3. Verificar la versión de Ubuntu

Antes de continuar es recomendable verificar la versión instalada.

Ejecutar:

```
lsb_release -a
```

Resultado esperado:

Distributor ID: Ubuntu

Description: Ubuntu 24.04.2 LTS

Release: 24.04

Codename: noble

### ¿Por qué se realiza esta verificación?

Confirma que el entorno corresponde a la versión de Ubuntu seleccionada para el proyecto y garantiza que las instrucciones de instalación sean compatibles.

## Instalación y configuración de Visual Studio Code

Visual Studio Code será el entorno de desarrollo integrado (IDE) utilizado para crear, editar y depurar los microservicios del Sistema de Gestión Hospitalaria. Gracias a su compatibilidad con Java, Go y Docker, permitirá trabajar con todos los componentes del proyecto desde una sola herramienta.

### Descargar Visual Studio Code

Descarga el paquete para Ubuntu desde el sitio oficial.

```
wget -O code.deb
```

```
https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64
```

### Instalar Visual Studio Code

Ejecuta:

```
sudo apt install ./code.deb -y
```



Una vez finalizada la instalación, verifica que Visual Studio Code esté disponible.

`code --version`

Si la instalación fue correcta, se mostrará la versión instalada.

## Instalar las extensiones

Abre Visual Studio Code.

`code`

Selecciona **Extensions** en el menú lateral o presiona **Ctrl + Shift + X**.

Instala las siguientes extensiones:

| Extensión                  | Uso                                                   |
|----------------------------|-------------------------------------------------------|
| Extension Pack for Java    | Desarrollo de microservicios con Spring Boot.         |
| Spring Boot Extension Pack | Desarrollo y administración de proyectos Spring Boot. |
| Go                         | Desarrollo de los microservicios implementados en Go. |
| Docker                     | Administración de contenedores Docker.                |
| YAML                       | Edición de archivos docker-compose.yml.               |
| GitLens                    | Administración y seguimiento de cambios en Git.       |

## Verificar el funcionamiento

Abre la carpeta del proyecto.

`code ~/SistemaGestionHospitalaria`

```
=====
Sistema de Gestión Hospitalaria
Instalación de Visual Studio Code
=====

Actualizando repositorios...
Obj:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Obj:2 http://mx.archive.ubuntu.com/ubuntu resolute InRelease
Obj:3 http://mx.archive.ubuntu.com/ubuntu resolute-updates InRelease
Obj:4 http://mx.archive.ubuntu.com/ubuntu resolute-backports InRelease
Se pueden actualizar 19 paquetes. Ejecute «apt list --upgradable» para verlos.

Instalando dependencias...
wget ya está en su versión más reciente (1.25.0-2ubuntu4.3).
Fijado wget como instalado manualmente.
gpg ya está en su versión más reciente (2.4.8-4ubuntu3).
Fijado gpg como instalado manualmente.
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
  linux-headers-7.0.0-14          linux-image-unsigned-7.0.0-14-generic  linux-modules-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-headers-7.0.0-14-generic  linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14
Utilice «sudo apt autoremove» para eliminarlos.

Actualizando:
  python3-software-properties  software-properties-common

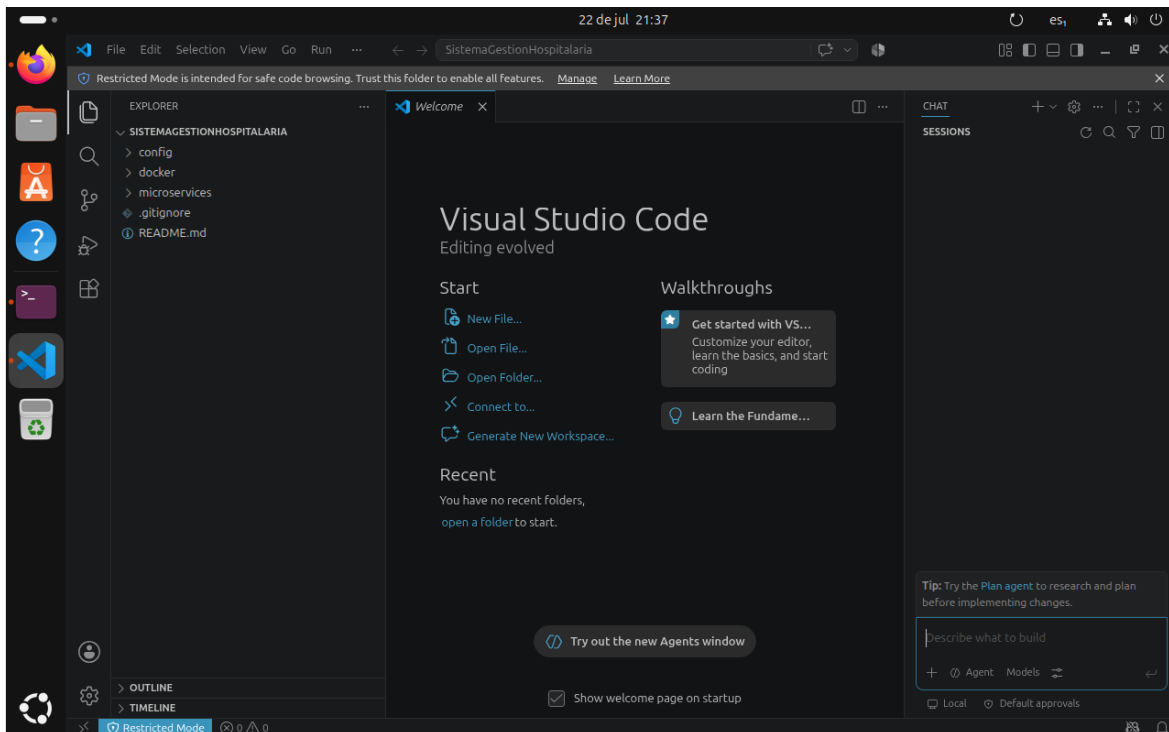
Instalando:
  apt-transport-https  curl

Resumen:
  Actualizando: 2, Instalando 2, Eliminando: 0, no actualizando: 17
  Tamaño de la descarga: 321 kB
  Espacio necesario: 559 kB / 38.4 GB disponible

Des:1 http://mx.archive.ubuntu.com/ubuntu resolute/universe amd64 apt-transport-https all 3.2.0 [3 928 B]
Des:2 http://mx.archive.ubuntu.com/ubuntu resolute-updates/main amd64 curl amd64 8.18.0-1ubuntu2.3 [272 kB]
Des:3 http://mx.archive.ubuntu.com/ubuntu resolute-updates/main amd64 software-properties-common all 0.120.1 [16.6 kB]
Des:4 http://mx.archive.ubuntu.com/ubuntu resolute-updates/main amd64 python3-software-properties all 0.120.1 [28.7 kB]
Descargados 321 kB en 2s (197 kB/s)
```

Visual Studio Code deberá mostrar la estructura del proyecto y reconocer automáticamente los archivos Java, Go y Docker Compose.

Al finalizar esta sección, Visual Studio Code quedará instalado y configurado como entorno de desarrollo para el Sistema de Gestión Hospitalaria. Además, contará con las extensiones necesarias para desarrollar los microservicios, administrar los contenedores Docker y trabajar con el repositorio Git.



## Instalación y configuración de Git

Git es un sistema de control de versiones distribuido que permitirá administrar el código fuente del Sistema de Gestión Hospitalaria. Gracias a esta herramienta será posible registrar cada cambio realizado en los microservicios, mantener un historial de versiones y sincronizar el proyecto con un repositorio remoto en GitHub.

### Paso 1. Verificar si Git ya está instalado

Antes de instalar Git, es recomendable comprobar si ya se encuentra disponible en el sistema.

Ejecuta el siguiente comando:

```
git --version
```

Si obtienes una respuesta similar a:  
git version 2.43.0

Git ya está instalado y puedes continuar con la configuración.  
Si aparece un mensaje indicando que el comando no existe, continúa con el siguiente paso.

## Paso 2. Instalar Git

Ejecuta el siguiente comando para actualizar la lista de paquetes disponibles en Ubuntu:

```
sudo apt update
```

Este comando consulta los repositorios configurados y obtiene la información más reciente de los paquetes disponibles.

Posteriormente instala Git ejecutando:

```
sudo apt install git -y
```

### Explicación del comando

- **sudo:** ejecuta el comando con permisos de administrador.
- **apt:** administrador de paquetes de Ubuntu.
- **install:** indica que se instalará un paquete.
- **git:** nombre del paquete que contiene el sistema de control de versiones.
- **-y:** acepta automáticamente la instalación sin solicitar confirmación.

Durante este proceso Ubuntu descargará e instalará Git junto con las dependencias necesarias para su funcionamiento.

## Paso 3. Verificar la instalación

Una vez finalizada la instalación, verifica que Git se haya instalado correctamente.

```
git --version
```

Deberás obtener una salida similar a:

```
git version 2.43.0
```

La versión puede variar dependiendo de las actualizaciones disponibles en los repositorios oficiales.

## Paso 4. Configurar el nombre del desarrollador

Git registra el nombre del autor en cada confirmación (*commit*). Configura el nombre ejecutando:

```
git config --global user.name "Pedro Cruz Rivas"
```

Sustituye "**Pedro Cruz Rivas**" por el nombre que desees asociar a tus cambios.

### **Paso 5. Configurar el correo electrónico**

Asocia el correo electrónico de tu cuenta de GitHub.

```
git config --global user.email "tu_correo@ejemplo.com"
```

Ejemplo:

```
git config --global user.email "pedrocruz@gmail.com"
```

Sustituye el correo de ejemplo por el que utilizas en GitHub.

### **Paso 6. Configurar la rama principal**

Configura **main** como la rama principal para los nuevos repositorios.

```
git config --global init.defaultBranch main
```

Esta configuración garantiza que todos los repositorios creados utilicen la rama **main**, siguiendo las prácticas actuales de desarrollo y manteniendo compatibilidad con GitHub.

### **Paso 7. Verificar la configuración**

Comprueba que la configuración se haya aplicado correctamente.

```
git config --list
```

El resultado deberá mostrar información similar a:

```
user.name=Pedro Cruz Rivas
user.email=pedrocruz@gmail.com
init.defaultBranch=main
```

Al finalizar esta sección, Git quedará correctamente instalado y configurado en Ubuntu Server. El entorno de desarrollo estará preparado para administrar el código fuente del Sistema de Gestión Hospitalaria, registrar el historial de cambios y sincronizar los repositorios con GitHub.

## Configuración de la plataforma de alojamiento del repositorio (GitHub)

GitHub será la plataforma utilizada para alojar el repositorio del Sistema de Gestión Hospitalaria. Permitirá almacenar el código fuente de los microservicios, controlar las versiones del proyecto y facilitar el trabajo colaborativo mediante Git.

### Crear un repositorio

Ingresa a <https://github.com> e inicia sesión con tu cuenta.

Selecciona la opción **New Repository**.

Asigna un nombre al repositorio, por ejemplo:

SistemaGestionHospitalaria

Selecciona la visibilidad del repositorio (**Public** o **Private**) y presiona **Create Repository**.

### Inicializar el repositorio local

Ubícate en la carpeta principal del proyecto.  
`cd ~/SistemaGestionHospitalaria`

Inicializa Git.  
`git init`

### Agregar los archivos del proyecto

Ejecuta:  
`git add .`

Este comando agrega todos los archivos del proyecto al área de preparación (*staging*).

### Crear el primer commit

Ejecuta:  
`git commit -m "Primer commit del proyecto"`

Con este comando se registra la primera versión del proyecto en el repositorio local.

## Vincular el repositorio remoto

Copia la URL del repositorio creado en GitHub y ejecútala en el siguiente comando:

```
git remote add origin https://github.com/usuario/SistemaGestionHospitalaria.git
```

Sustituye **usuario** por el nombre de tu cuenta de GitHub.

## Enviar el proyecto a GitHub

Ejecuta:

```
git branch -M main
```

Posteriormente envía el proyecto al repositorio remoto.

```
git push -u origin main
```

Si la autenticación es correcta, el proyecto quedará almacenado en GitHub.

## Verificar la conexión

Comprueba que el repositorio remoto quedó asociado correctamente.

```
git remote -v
```

El resultado deberá mostrar una salida similar a:

```
origin https://github.com/usuario/SistemaGestionHospitalaria.git (fetch)
```

```
origin https://github.com/usuario/SistemaGestionHospitalaria.git (push)
```

## Resultado esperado

Al finalizar esta sección, el código fuente del Sistema de Gestión Hospitalaria quedará almacenado en GitHub y podrá sincronizarse mediante Git para registrar nuevas versiones y respaldar el desarrollo del proyecto.

## Instalación y configuración de OpenJDK 21 LTS

Los microservicios **MS Consulta** y **MS EHR** fueron desarrollados con **Spring Boot**, por lo que es necesario instalar un entorno de desarrollo Java. Para este proyecto se utilizará **OpenJDK 21 LTS**, ya que ofrece estabilidad y compatibilidad con Spring Boot.

## **Paso 1. Verificar si Java está instalado**

Ejecuta el siguiente comando:

```
java -version
```

Si el sistema muestra la versión instalada, puedes continuar con la siguiente sección. En caso contrario, instala OpenJDK.

## **Paso 2. Instalar OpenJDK 21**

Ejecuta:

```
sudo apt install openjdk-21-jdk -y
```

Este comando descarga e instala el Kit de Desarrollo de Java (JDK), incluyendo el compilador (javac), el entorno de ejecución (JRE) y la Máquina Virtual de Java (JVM).

## **Paso 3. Verificar la instalación**

Comprueba que Java se instaló correctamente.

```
java -version
```

También verifica el compilador.

```
javac -version
```

Si ambos comandos muestran la versión instalada, Java quedó correctamente configurado.

## **Paso 4. Configurar la variable JAVA\_HOME**

Obtén la ruta donde fue instalado Java.

```
readlink -f $(which java)
```

El resultado será similar a:

```
/usr/lib/jvm/java-21-openjdk-amd64/bin/java
```

Abre el archivo de variables de entorno.

```
sudo nano /etc/environment
```

Agrega la siguiente línea:

```
JAVA_HOME="/usr/lib/jvm/java-21-openjdk-amd64"
```

Guarda los cambios y cierra el editor.

### **Paso 5. Aplicar los cambios**

Ejecuta:

```
source /etc/environment
```

Verifica que la variable fue creada correctamente.

```
echo $JAVA_HOME
```

Deberá mostrarse la ruta del JDK.

Al finalizar esta sección, Ubuntu contará con un entorno Java completamente funcional, permitiendo compilar y ejecutar los microservicios **MS Consulta** y **MS EHR** desarrollados con Spring Boot.

### **Instalación y configuración de Apache Maven**

Después de OpenJDK 21, el siguiente componente de tu arquitectura es Apache Maven, ya que los microservicios MS Consulta y MS EHR fueron desarrollados con Spring Boot y utilizan Maven para administrar dependencias y generar los archivos.jar.

Apache Maven es una herramienta de automatización utilizada para gestionar las dependencias, compilar el código fuente y generar los archivos ejecutables de los microservicios desarrollados con Spring Boot.

### **Paso 1. Verificar si Maven está instalado**

Ejecuta el siguiente comando:

```
mvn -version
```

Si el sistema muestra la versión instalada, puedes continuar con la siguiente sección. En caso contrario, instala Maven.



## **Paso 2. Instalar Apache Maven**

Ejecuta:

```
sudo apt install maven -y
```

Este comando instala Apache Maven y las bibliotecas necesarias para compilar proyectos Java.

## **Paso 3. Verificar la instalación**

Comprueba que Maven se instaló correctamente.

```
mvn -version
```

**El resultado será similar al siguiente:**

```
Apache Maven 3.9.x  
Maven home: /usr/share/maven  
Java version: 21
```

## **Paso 4. Verificar la integración con Java**

Maven utiliza el JDK instalado previamente. Para comprobar que reconoce la instalación de Java, ejecuta nuevamente:

```
mvn -version
```

Verifica que en la salida aparezca una línea similar a:

```
Java version: 21
```

Esto confirma que Maven utilizará OpenJDK 21 para compilar los proyectos.

## Paso 5. Probar Maven

Crea un proyecto de prueba para comprobar que la instalación funciona correctamente.

```
mvn archetype:generate \  
-DgroupId=com.hospital \  
-DartifactId=proyecto-prueba \  
-DarchetypeArtifactId=maven-archetype-quickstart \  
-DinteractiveMode=false
```

Este comando genera un proyecto Java básico utilizando uno de los arquetipos incluidos en Maven.

Una vez creado, ingresa al proyecto:

```
cd proyecto-prueba
```

Compílalo ejecutando:

```
mvn clean package
```

Si el proceso finaliza correctamente, Maven generará un archivo .jar dentro de la carpeta target, confirmando que el entorno está listo para compilar los microservicios del proyecto.

Al finalizar esta sección, Apache Maven quedará instalado y configurado correctamente. El entorno estará preparado para descargar las dependencias de Spring Boot, compilar los microservicios MS Consulta y MS EHR, ejecutar pruebas y generar los archivos .jar que posteriormente serán empaquetados en contenedores Docker.

## **Instalación y configuración de Go**

Go es el lenguaje de programación utilizado para desarrollar los microservicios MS Turnos y MS Farmacia. Su instalación permitirá compilar, ejecutar y administrar las aplicaciones desarrolladas para este proyecto.

### **Paso 1. Verificar si Go está instalado**

Ejecuta el siguiente comando:  
`go version`

Si el sistema muestra una versión instalada, puedes continuar con la siguiente sección. En caso contrario, instala Go.

### **Paso 2. Descargar Go**

Descarga la versión estable desde el sitio oficial:

`wget https://go.dev/dl/go1.24.5.linux-amd64.tar.gz`

Este comando descarga el paquete de instalación de Go para sistemas Linux de 64 bits.

### **Paso 3. Eliminar instalaciones anteriores (si existen)**

`sudo rm -rf /usr/local/go`

Este comando elimina una instalación previa para evitar conflictos con la nueva versión.

#### **Paso 4. Instalar Go**

Extrae el paquete descargado en el directorio /usr/local.

```
sudo tar -C /usr/local -xzf go1.24.5.linux-amd64.tar.gz
```

#### **Paso 5. Configurar la variable PATH**

Abre el archivo de configuración del usuario.

```
nano ~/.profile
```

**Agrega al final la siguiente línea:**

```
export PATH=$PATH:/usr/local/go/bin
```

Guarda los cambios y cierra el editor.

#### **Paso 6. Aplicar los cambios**

Ejecuta:

```
source ~/.profile
```

Con esto, el sistema reconocerá el comando go desde cualquier ubicación.

#### **Paso 7. Verificar la instalación**

Comprueba que Go quedó instalado correctamente.

```
go version
```

El resultado será similar a:

```
go version go1.24.5 linux/amd64
```

#### **Paso 8. Verificar la ubicación de Go**

Ejecuta:

```
which go
```

El resultado esperado es:

```
/usr/local/go/bin/go
```

Al finalizar esta sección, Go quedará instalado y configurado correctamente en Ubuntu Server. El entorno estará preparado para compilar y ejecutar los microservicios MS Turnos y MS Farmacia, desarrollados con el framework Gin, así como para gestionar sus dependencias mediante los módulos de Go.

## **Instalación y configuración de Docker Engine**

Docker es una plataforma de virtualización basada en contenedores que permitirá ejecutar los microservicios y los servicios de apoyo del Sistema de Gestión Hospitalaria de forma aislada, portable y consistente en cualquier entorno.

Verificar si Docker está instalado

Ejecuta:  
`docker --version`

Si el sistema muestra una versión instalada, puedes continuar con la siguiente sección. En caso contrario, realiza la instalación.

### **Agregar el repositorio oficial de Docker**

Instala los paquetes necesarios para agregar el repositorio oficial.

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common  
-y
```

### **Importa la llave GPG de Docker.**

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o  
/usr/share/keyrings/docker-archive-keyring.gpg
```

### **Agrega el repositorio oficial.**

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-  
archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release  
&& echo $VERSION_CODENAME) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

### **Actualiza nuevamente los repositorios.**

```
sudo apt update  
Instalar Docker
```

Ejecuta:

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-  
compose-plugin -y
```

Este comando instala Docker Engine, la interfaz de línea de comandos, el motor de ejecución de contenedores, Buildx y el complemento oficial de Docker Compose.

## **Verificar la instalación**

Comprueba que Docker quedó instalado correctamente.

```
docker --version
```

## **También verifica el estado del servicio.**

```
sudo systemctl status docker
```

El servicio deberá aparecer con el estado active (running).

Permitir ejecutar Docker sin sudo

Agrega el usuario actual al grupo de Docker.

```
sudo usermod -aG docker $USER
```

## **Aplica el cambio iniciando nuevamente la sesión o ejecutando:**

```
newgrp docker  
Probar Docker
```

Ejecuta el siguiente contenedor de prueba.

```
docker run hello-world
```

Si Docker está funcionando correctamente, aparecerá un mensaje indicando que la instalación fue exitosa.

## **Resultado esperado**

Al finalizar esta sección, Docker Engine quedará instalado y listo para ejecutar los contenedores del Sistema de Gestión Hospitalaria. Sobre esta plataforma se desplegarán los microservicios desarrollados en Spring Boot y Go, además de PostgreSQL, MongoDB, Redis, Kafka, ZooKeeper, Kong Gateway y los demás componentes definidos en la arquitectura del proyecto.

## **Verificación de Docker Compose**

Docker Compose permite definir y ejecutar aplicaciones compuestas por múltiples contenedores mediante un archivo `docker-compose.yml`. En este proyecto se utilizará para desplegar los microservicios y los servicios de infraestructura, como PostgreSQL, MongoDB, Redis, Kafka, ZooKeeper y Kong Gateway.

### **Verificar la instalación**

Ejecuta el siguiente comando:  
`docker compose version`

Si Docker Compose está instalado correctamente, se mostrará un resultado similar al siguiente:

`Docker Compose version v2.x.x`

### **Probar el funcionamiento**

Verifica que el comando esté disponible mostrando su ayuda.

`docker compose --help`

Si el comando responde mostrando las opciones disponibles, Docker Compose quedó listo para utilizarse.

### **Crear un directorio para el proyecto**

Crea la carpeta donde se almacenarán los archivos del sistema.

`mkdir SistemaGestionHospitalaria`

### **Ingresa al directorio.**

`cd SistemaGestionHospitalaria`

### **Crea el archivo principal de Docker Compose.**

`touch docker-compose.yml`

Este archivo contendrá la definición de todos los contenedores utilizados por el proyecto.

Al finalizar esta sección, Docker Compose estará listo para orquestar el despliegue de los microservicios MS Consulta, MS EHR, MS Turnos y MS Farmacia, así como de PostgreSQL, MongoDB, Redis, Kafka, ZooKeeper y Kong Gateway definidos en la arquitectura del Sistema de Gestión Hospitalaria.

## **Creación de la estructura del proyecto**

En esta sección se crea la estructura base del Sistema de Gestión Hospitalaria. Esta organización facilitará el desarrollo de los microservicios, la administración de los archivos de configuración y el despliegue mediante Docker Compose.

Crear el directorio principal

Ejecuta:

```
mkdir SistemaGestionHospitalaria
```

Ingresa al directorio.

```
cd SistemaGestionHospitalaria
```

### **Crear la estructura de carpetas**

**Ejecuta los siguientes comando:**

```
mkdir -p microservices/ms-consulta \  
microservices/ms-ehr \  
microservices/ms-turnos \  
microservices/ms-farmacia \  
config/kong \  
config/kafka \  
config/postgresql \  
config/mongodb \  
config/redis \  
config/zookeeper \  
docker
```

Este comando crea las carpetas donde se almacenarán los microservicios y los archivos de configuración de cada servicio.

### **Verificar la estructura creada**

Ejecuta:

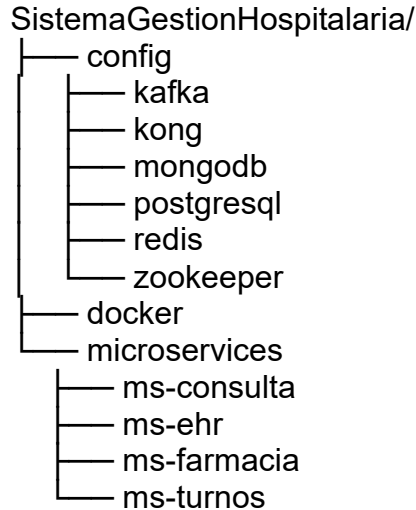
```
tree
```

Si el comando no está disponible, instálalo:

```
sudo apt install tree -y
```



La estructura deberá ser similar a la siguiente:



Al finalizar esta sección, el proyecto contará con una estructura organizada para almacenar los microservicios y la configuración de los servicios de infraestructura. Esta organización servirá como base para la creación del archivo `docker-compose.yml` y el despliegue de toda la arquitectura del Sistema de Gestión Hospitalaria.

### **Creación del archivo Docker Compose**

Docker Compose permitirá definir y administrar todos los contenedores del Sistema de Gestión Hospitalaria desde un único archivo denominado `docker-compose.yml`. En este archivo se configurarán los microservicios y los servicios de infraestructura que conforman la arquitectura del proyecto.

### **Crear el archivo Docker Compose**

Ubícate en el directorio principal del proyecto.

```
cd ~/SistemaGestionHospitalaria
```

Crea el archivo de configuración.

```
touch docker-compose.yml
```

Editar el archivo

Abre el archivo con el editor Nano.

```
nano docker-compose.yml
```

A partir de este archivo se agregarán las configuraciones correspondientes a cada uno de los servicios del sistema.

Verificar la creación del archivo

### **Ejecuta:**

```
ls -l
```

Deberá mostrarse un resultado similar al siguiente:

```
docker-compose.yml
config/
docker/
microservices/
Validar la sintaxis
```

Una vez que el archivo contenga la configuración de los servicios, podrá verificarse con el siguiente comando:

```
docker compose config
```

Si no existen errores de sintaxis, Docker Compose mostrará la configuración procesada sin generar mensajes de error.

Al finalizar esta sección, el proyecto contará con el archivo `docker-compose.yml`, el cual servirá como punto central para definir y administrar todos los contenedores del Sistema de Gestión Hospitalaria.

El siguiente apartado será la configuración de los servicios dentro del archivo `docker-compose.yml`, comenzando con PostgreSQL, que será la base de datos relacional utilizada por los microservicios desarrollados con Spring Boot y Go.

## Configuración de PostgreSQL en Docker Compose

PostgreSQL será la base de datos relacional del Sistema de Gestión Hospitalaria. Se ejecutará en un contenedor Docker y almacenará la información utilizada por los microservicios MS Consulta y MS EHR.

### Editar el archivo Docker Compose

Ubícate en la carpeta principal del proyecto.

```
cd ~/SistemaGestionHospitalaria
```

Abre el archivo de configuración.  
`nano docker-compose.yml`

Agrega la siguiente configuración:

services:

postgres:

```
image: postgres:17
container_name: postgres-hospital
restart: always
```

environment:

```
POSTGRES_DB: hospitaldb
POSTGRES_USER: hospital
POSTGRES_PASSWORD: hospital123
```

ports:

```
- "5432:5432"
```

volumes:

```
- postgres_data:/var/lib/postgresql/data
```

volumes:

postgres\_data:

Guarda los cambios y cierra el editor.

Verificar la sintaxis

### Ejecuta:

```
docker compose config
```

Si la configuración es correcta, Docker Compose mostrará el contenido procesado sin errores.

### **Iniciar el contenedor**

Ejecuta:

```
docker compose up -d postgres
```

**Docker descargará la imagen oficial de PostgreSQL y creará el contenedor.**

Verificar el contenedor

Comprueba que el contenedor se encuentra en ejecución.

```
docker ps
```

**El resultado deberá mostrar un contenedor similar al siguiente:**

| CONTAINER ID | IMAGE       | NAME              |
|--------------|-------------|-------------------|
| xxxxxxxxxxxx | postgres:17 | postgres-hospital |

Verificar los registros

Para confirmar que PostgreSQL inició correctamente, ejecuta:

```
docker logs postgres-hospital
```

Entre los mensajes deberá aparecer uno similar a:

```
database system is ready to accept connections
```

Al finalizar esta sección, PostgreSQL estará ejecutándose en un contenedor Docker y listo para ser utilizado por los microservicios del Sistema de Gestión Hospitalaria.

## Configuración de MongoDB en Docker Compose

MongoDB es una base de datos NoSQL orientada a documentos. En este proyecto se ejecutará como un contenedor Docker y complementará a PostgreSQL para el almacenamiento de información que no requiere un modelo relacional.

### Editar el archivo Docker Compose

Abre el archivo de configuración.

```
nano ~/SistemaGestionHospitalaria/docker-compose.yml
```

Agrega la siguiente configuración debajo del servicio PostgreSQL.

```
mongodb:
  image: mongo:8
  container_name: mongodb-hospital
  restart: always

  environment:
    MONGO_INITDB_ROOT_USERNAME: admin
    MONGO_INITDB_ROOT_PASSWORD: admin123

  ports:
    - "27017:27017"

  volumes:
    - mongodb_data:/data/db
```

En la sección volumes, agrega:

```
mongodb_data:
```

### Guarda los cambios y cierra el editor.

### Verificar la configuración

Ejecuta:

```
docker compose config
```

Si no existen errores, la configuración es válida.  
Iniciar el contenedor

Ejecuta:

```
docker compose up -d mongodb
```

Docker descargará la imagen oficial de MongoDB y creará el contenedor.

Verificar el funcionamiento

Comprueba que el contenedor se encuentra en ejecución.

```
docker ps
```

Deberá aparecer un resultado similar al siguiente:

| CONTAINER ID | IMAGE   | NAME             |
|--------------|---------|------------------|
| xxxxxxxxxxxx | mongo:8 | mongodb-hospital |

También puedes revisar los registros del servicio.

```
docker logs mongodb-hospital
```

Si no se muestran errores, MongoDB inició correctamente.

Al finalizar esta sección, MongoDB estará disponible como un contenedor Docker y listo para ser utilizado por los microservicios del Sistema de Gestión Hospitalaria.

## **Configuración de ZooKeeper en Docker Compose**

ZooKeeper es un servicio de coordinación distribuida utilizado por Apache Kafka para gestionar los brokers y mantener el estado del clúster. En este proyecto se ejecutará como un contenedor Docker para soportar la comunicación asíncrona entre los microservicios.

Editar el archivo Docker Compose

Abre el archivo de configuración.

```
nano ~/SistemaGestionHospitalaria/docker-compose.yml
```

### **Agrega el siguiente servicio debajo de Redis.**

```
zookeeper:  
  image: confluentinc/cp-zookeeper:7.7.0  
  container_name: zookeeper-hospital  
  restart: always
```

```
environment:  
  ZOOKEEPER_CLIENT_PORT: 2181  
  ZOOKEEPER_TICK_TIME: 2000
```

ports:  
- "2181:2181"

**Guarda los cambios y cierra el editor.**

**Verificar la configuración**

Ejecuta:

`docker compose config`

Si no se muestran errores, la configuración es válida.

**Iniciar el contenedor**

Ejecuta:

`docker compose up -d zookeeper`

**Docker descargará la imagen y creará el contenedor de ZooKeeper.**

Verificar el funcionamiento

Comprueba que el contenedor está en ejecución.

`docker ps`

**Deberá aparecer un resultado similar al siguiente:**

| CONTAINER ID | IMAGE                           | NAME               |
|--------------|---------------------------------|--------------------|
| xxxxxxxxxxxx | confluentinc/cp-zookeeper:7.7.0 | zookeeper-hospital |

También puedes revisar los registros del servicio.

`docker logs zookeeper-hospital`

Si no aparecen mensajes de error, ZooKeeper inició correctamente.

Al finalizar esta sección, ZooKeeper estará ejecutándose en un contenedor Docker y listo para administrar el clúster de Apache Kafka utilizado por los microservicios del Sistema de Gestión Hospitalaria.

## Configuración de Apache Kafka en Docker Compose

Apache Kafka es una plataforma de mensajería distribuida que permitirá la comunicación entre los microservicios del Sistema de Gestión Hospitalaria mediante eventos, reduciendo el acoplamiento entre los servicios.

Editar el archivo Docker Compose

Abre el archivo de configuración.

```
nano ~/SistemaGestionHospitalaria/docker-compose.yml
```

Agrega el siguiente servicio debajo de ZooKeeper.

```
kafka:
  image: confluentinc/cp-kafka:7.7.0
  container_name: kafka-hospital
  restart: always

  depends_on:
    - zookeeper

  ports:
    - "9092:9092"

  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Guarda los cambios y cierra el editor.

Verificar la configuración

Ejecuta:

```
docker compose config
```

Si no se muestran errores, la configuración es correcta.

Iniciar el contenedor

Ejecuta:

```
docker compose up -d kafka
```



**Docker descargará la imagen oficial de Apache Kafka y creará el contenedor.**

Verificar el funcionamiento

**Comprueba que el contenedor está en ejecución.**

`docker ps`

**Deberá aparecer un resultado similar al siguiente:**

| CONTAINER ID | IMAGE                       | NAME           |
|--------------|-----------------------------|----------------|
| xxxxxxxxxxxx | confluentinc/cp-kafka:7.7.0 | kafka-hospital |

También puedes revisar los registros del servicio.

`docker logs kafka-hospital`

Si no aparecen mensajes de error, Kafka inició correctamente.

Al finalizar esta sección, Apache Kafka estará ejecutándose en un contenedor Docker y listo para gestionar la comunicación asíncrona entre los microservicios del Sistema de Gestión Hospitalaria.

## **Configuración de Kong Gateway en Docker Compose**

Kong Gateway es un API Gateway que centraliza las solicitudes dirigidas a los microservicios. En este proyecto permitirá administrar el enrutamiento de peticiones, facilitando el acceso a los servicios desarrollados con Spring Boot y Go.

### **Editar el archivo Docker Compose**

Abre el archivo de configuración.

`nano ~/SistemaGestionHospitalaria/docker-compose.yml`

## **Agrega el siguiente servicio debajo de Kafka.**

```
kong:
  image: kong:3.7
  container_name: kong-hospital
  restart: always

  environment:
    KONG_DATABASE: "off"
    KONG_DECLARATIVE_CONFIG: /opt/kong/kong.yml
    KONG_PROXY_ACCESS_LOG: /dev/stdout
    KONG_ADMIN_ACCESS_LOG: /dev/stdout
    KONG_PROXY_ERROR_LOG: /dev/stderr
    KONG_ADMIN_ERROR_LOG: /dev/stderr
    KONG_ADMIN_LISTEN: 0.0.0.0:8001

  ports:
    - "8000:8000"
    - "8001:8001"

  volumes:
    - ./config/kong/kong.yml:/opt/kong/kong.yml
```

Guarda los cambios y cierra el editor.

## **Crear el archivo de configuración de Kong**

Crea el archivo donde se definirán las rutas hacia los microservicios.

```
touch ~/SistemaGestionHospitalaria/config/kong/kong.yml
```

Verificar la configuración

Ejecuta:

```
docker compose config
```

Si no se muestran errores, la configuración es válida.

Iniciar el contenedor

Ejecuta:

```
docker compose up -d kong
```

Verificar el funcionamiento

Comprueba que el contenedor está en ejecución.

```
docker ps
```

**Deberá mostrarse un resultado similar al siguiente:**

```
CONTAINER ID  IMAGE      NAME
xxxxxxxxxxxx  kong:3.7   kong-hospital
```

También puedes consultar los registros del servicio.

```
docker logs kong-hospital
```

Si no aparecen errores, Kong Gateway quedó listo para utilizarse.

Al finalizar esta sección, Kong Gateway estará configurado como el punto de entrada de la arquitectura del Sistema de Gestión Hospitalaria. Posteriormente se definirán las rutas que dirigirán las solicitudes hacia los microservicios correspondientes.

## **Despliegue de la infraestructura con Docker Compose**

Una vez configurados los servicios de PostgreSQL, MongoDB, Redis, ZooKeeper, Kafka y Kong Gateway, el siguiente paso consiste en desplegar toda la infraestructura utilizando Docker Compose.

Iniciar los servicios

Desde la carpeta principal del proyecto, ejecuta:

```
cd ~/SistemaGestionHospitalaria
```

Levanta todos los contenedores definidos en el archivo docker-compose.yml.  
`docker compose up -d`

Docker descargará automáticamente las imágenes necesarias y creará los contenedores definidos en la configuración.

Verificar los contenedores

Comprueba que todos los servicios se encuentren en ejecución.

```
docker ps
```

**El resultado deberá mostrar los siguientes contenedores:**

postgres-hospital  
mongodb-hospital  
redis-hospital  
zookeeper-hospital  
kafka-hospital  
kong-hospital  
Verificar el estado de los servicios

Ejecuta:  
docker compose ps

**Todos los servicios deberán aparecer con el estado running o Up.**

Consultar los registros

**Si algún servicio presenta problemas durante el inicio, consulta los registros con:**

docker compose logs

**También es posible consultar un servicio específico.**

docker logs postgres-hospital  
Detener la infraestructura

**Cuando sea necesario detener los servicios, ejecuta:**

docker compose down

Este comando detendrá todos los contenedores sin eliminar las imágenes descargadas.

Al finalizar esta sección, toda la infraestructura del Sistema de Gestión Hospitalaria estará desplegada y en ejecución mediante Docker Compose. Los servicios de PostgreSQL, MongoDB, Redis, ZooKeeper, Kafka y Kong Gateway quedarán disponibles para ser utilizados por los microservicios.

## Integración de los microservicios

Una vez desplegada la infraestructura, se incorporan los microservicios del Sistema de Gestión Hospitalaria al archivo `docker-compose.yml`. Cada servicio se ejecutará en un contenedor independiente y podrá comunicarse con los demás componentes de la arquitectura.

Agregar los microservicios

Abre el archivo de configuración.

```
nano ~/SistemaGestionHospitalaria/docker-compose.yml
```

Agrega los servicios correspondientes a los cuatro microservicios del proyecto.

```
ms-consulta:
  build: ./microservices/ms-consulta
  container_name: ms-consulta
  restart: always
  depends_on:
    - postgres
```

```
ms-ehr:
  build: ./microservices/ms-ehr
  container_name: ms-ehr
  restart: always
  depends_on:
    - postgres
    - mongodb
```

```
ms-turnos:
  build: ./microservices/ms-turnos
  container_name: ms-turnos
  restart: always
  depends_on:
    - redis
    - kafka
```

```
ms-farmacia:
  build: ./microservices/ms-farmacia
  container_name: ms-farmacia
  restart: always
  depends_on:
    - postgres
    - kafka
```

Guarda los cambios y cierra el editor.

Verificar la configuración

**Comprueba que la sintaxis del archivo sea correcta.**

`docker compose config`

**Si no se muestran errores, la configuración es válida.**

Construir los microservicios

Genera las imágenes Docker de los microservicios.

`docker compose build`

Docker utilizará los archivos Dockerfile de cada microservicio para crear las imágenes correspondientes.

**Iniciar los microservicios**

Ejecuta:

`docker compose up -d`

**Los cuatro microservicios se iniciarán junto con los servicios de infraestructura.**

Verificar el despliegue

**Comprueba que todos los contenedores se encuentren en ejecución.**

`docker ps`

Deberá mostrarse una lista similar a la siguiente:

postgres-hospital  
mongodb-hospital  
redis-hospital  
zookeeper-hospital  
kafka-hospital  
kong-hospital  
ms-consulta  
ms-ehr  
ms-turnos  
ms-farmacia  
Resultado esperado

Al finalizar esta sección, los cuatro microservicios quedarán integrados a la arquitectura del Sistema de Gestión Hospitalaria y estarán disponibles para comunicarse con las bases de datos, los servicios de mensajería y el API Gateway.

### **Verificación de la comunicación entre microservicios**

Una vez desplegados los microservicios, es necesario verificar que la comunicación mediante REST, gRPC y Apache Kafka funcione correctamente, garantizando el intercambio de información entre los componentes del Sistema de Gestión Hospitalaria.

#### **Verificar la red de Docker**

Ejecuta el siguiente comando para comprobar que todos los contenedores pertenecen a la misma red.

```
docker network ls
```

#### **Identifica la red creada por Docker Compose y consulta su configuración.**

```
docker network inspect sistemagesthospitalaria_default
```

En el resultado deberán aparecer los contenedores de PostgreSQL, MongoDB, Redis, ZooKeeper, Kafka, Kong Gateway y los cuatro microservicios.

#### **Verificar la comunicación REST**

Comprueba que los servicios respondan mediante HTTP.

```
curl http://localhost:8080/actuator/health
```

Si el microservicio está disponible, se obtendrá una respuesta similar a:

```
{
  "status": "UP"
}
```

Verificar la comunicación gRPC

#### **Consulta los registros del microservicio que utiliza gRPC.**

```
docker logs ms-consulta
```

Si la comunicación es correcta, no deberán aparecer errores relacionados con la conexión al servidor gRPC.

## **Verificar la comunicación con Kafka**

Consulta los registros del contenedor.

```
docker logs kafka-hospital
```

También verifica los registros de los microservicios consumidores.

```
docker logs ms-turnos
```

```
docker logs ms-farmacia
```

No deberán aparecer errores relacionados con la conexión al broker de Kafka.

Verificar el acceso a las bases de datos

## **Comprueba que los microservicios puedan conectarse a PostgreSQL.**

```
docker logs ms-consulta
```

Verifica también la conexión con MongoDB.

```
docker logs ms-ehr
```

Si la configuración es correcta, los registros no mostrarán errores de conexión.

Al finalizar esta sección, se habrá comprobado que los microservicios pueden comunicarse correctamente utilizando REST, gRPC y Apache Kafka, además de acceder a los servicios de PostgreSQL, MongoDB y Redis definidos en la arquitectura del Sistema de Gestión Hospitalaria.



# Conclusiones

1. Como equipo, concluimos que la correcta configuración del entorno de desarrollo constituye una de las etapas más importantes para el éxito del proyecto. La instalación y configuración de herramientas como Ubuntu Server, Visual Studio Code, Git, GitHub, OpenJDK, Apache Maven, Go, Docker Engine y Docker Compose nos permitió establecer una plataforma de trabajo estandarizada y preparada para el desarrollo de los microservicios del Sistema de Gestión Hospitalaria. Esto garantiza que todos los integrantes trabajen sobre un mismo entorno, reduciendo problemas de compatibilidad y facilitando la integración del código.
2. Consideramos que una de las principales fortalezas del prototipo es la preparación de una infraestructura tecnológica basada en contenedores, la cual incorpora PostgreSQL, MongoDB, Redis, Apache Kafka, ZooKeeper y Kong Gateway como componentes de apoyo para la arquitectura distribuida. La integración de estas tecnologías mediante Docker Compose proporciona un entorno organizado, reproducible y escalable, permitiendo que los servicios puedan desplegarse de manera coordinada y facilitando futuras tareas de mantenimiento y expansión del sistema.
3. Durante el desarrollo de esta actividad identificamos que la configuración de un entorno distribuido requiere una adecuada planificación y una correcta integración entre las distintas herramientas tecnológicas. La administración del repositorio mediante Git y GitHub, la definición de una estrategia de ramas, la organización de la estructura de carpetas y la preparación de la infraestructura representan elementos fundamentales para asegurar un proceso de desarrollo colaborativo, ordenado y alineado con las buenas prácticas de la ingeniería de software.
4. Como limitación del prototipo, reconocemos que esta etapa estuvo enfocada principalmente en la preparación del entorno de desarrollo y de la infraestructura tecnológica, por lo que la implementación funcional de los microservicios, las pruebas de integración, las validaciones de rendimiento y los mecanismos de seguridad aún deberán desarrollarse en las siguientes fases del proyecto. No obstante, contar con un entorno previamente configurado permitirá reducir tiempos de implementación y facilitar la incorporación de nuevas funcionalidades conforme avance el desarrollo de la solución.
5. Finalmente, como equipo consideramos que el trabajo realizado proporciona una base sólida para continuar con las siguientes etapas del Sistema de Gestión Hospitalaria. Entre las mejoras previstas se encuentran la implementación de la lógica de negocio de cada microservicio, la integración completa entre los componentes mediante REST, gRPC y Apache Kafka, la incorporación de mecanismos de autenticación y autorización, así como la ejecución de pruebas funcionales y de desempeño que permitan validar el comportamiento de la plataforma antes de su despliegue en un entorno de producción.